

State Educational Institution
“Specialized School-Lyceum No. 54”
named after I.V. Panfilov”

Neuralese: Educational Platform for AI System Development Training

Completed by:
Isakov Mikhail, Grade 10
Nurmukhanbetov Rakhim, Grade 10

Supervisor:
Dina Amirkanova

Almaty, 2026

Abstract

Artificial intelligence education faces a dual challenge: tools that provide accessibility limit control over neural network design, while flexible frameworks impose significant programming and computational demands. This tradeoff prevents students from gaining practical experience in building and training models.

This project develops Neuralese, a platform enabling students to construct, train, and deploy neural networks through a visual-interactive environment without programming. The system supports the full AI development workflow, including model construction, dataset handling, interactive training, and deployment.

The platform uses a client-server architecture with GPU-accelerated training. Two optimizations support concurrent training workloads: Section Reuse, which exploits architectural similarity between models, and Fused SuperGraph execution, which consolidates operations across training sessions.

Evaluation with 83 students showed higher learning gains than traditional instruction. Grade 7 and Grade 5 experimental groups improved by 18% and 10% respectively compared to controls (Cliff's $\delta \approx 0.56$ and 0.42). System optimizations reduced training time for 16 users by up to $3.4\times$ and kernel launches by approximately $5\times$.

These results indicate that visual-interactive neural network development can improve accessibility while supporting scalable AI education.

Table of contents

Introduction	1
Research section	3
1) Review of Existing Solutions	3
2) Technical Challenge of Scalability in Educational AI Platforms	8
3) Platform Architecture of Neuralese	9
3.1) System Overview	9
3.2) Development process and high-level technologies	10
3.3) Node graph system	11
3.4) Dataset editor	11
3.5) Simulations as evaluation environments	12
3.6) AI Mentor Architecture: Axon	13
3.7) Structured Lesson System	15
3.8) Model Export: Yolk-Protein	16
4) Initial pilot study	17
4.1) Pilot Lesson Design and Results	17
4.2) Limitations Identified and Engineering Consequences	17
5) Algorithmic Optimizations for Concurrent Training	18
5.1) Section Reuse: Exploiting Architectural Similarity	18
5.2) Fused SuperGraph: Consolidated Execution	21
6) Pedagogical validation	23
6.1) Study Design and Implementation	23
6.2) Results and Effect Sizes	25
6.3) Interpretation	26
Conclusion	27
References	29

Introduction

The AI Education Crisis and Infrastructure Challenge

Artificial intelligence has evolved from a specialized research domain to critical infrastructure underlying medical diagnosis, autonomous systems, financial markets, industrial automation, and scientific discovery. The economic impact of AI deployment continues to accelerate, with the global AI market growing from approximately 20 billion USD in 2018 to over 200 billion USD by early 2026, representing nearly ninefold expansion in less than a decade (see Figure 1). This rapid integration of AI technologies across sectors generates unprecedented demand for specialists capable of developing, deploying, and maintaining neural network systems. ^[1]

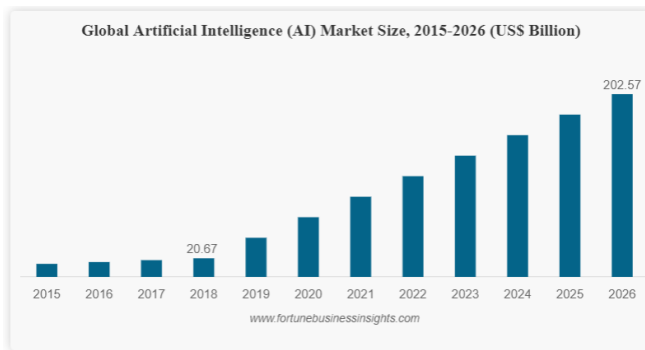


Figure 1. AI market dynamics. Obtained from fortunebusinessinsights.com, 2025

Current workforce analyses indicate severe talent shortages constraining AI adoption. Industry estimates suggest global demand for AI specialists exceeds supply by approximately threefold, substantially higher than observed in most other technical domains. This disparity reflects not merely quantitative insufficiency but a fundamental mismatch between the skills employers require and those educational systems produce. Organizations seek practitioners capable of independent model development, architectural decision-making, and deployment of systems addressing novel problems. Educational institutions struggle to deliver these capabilities, particularly at secondary and early tertiary levels where foundational skill acquisition occurs.

The educational technology landscape exhibits a structural discontinuity preventing effective skill development. Tools designed for accessibility sacrifice technical depth, reducing neural network construction to parameter selection within predefined templates. Users observe model behavior without understanding architectural principles or developing diagnostic capabilities required for independent work. Conversely, professional development frameworks provide comprehensive control over model design and training procedures but demand proficiency in programming languages, linear algebra, calculus, and optimization theory. The cognitive load imposed by simultaneous management of code syntax, mathematical abstractions, and machine learning concepts exceeds the capacity of learners lacking substantial prior technical preparation.

This creates a paradoxical situation where AI literacy initiatives proliferate while practical development skills remain concentrated among those with advanced technical backgrounds. Secondary school students learn to recognize AI applications and discuss societal implications but acquire minimal capability to construct functioning systems. University programs successfully train

students with strong quantitative foundations but remain inaccessible to those without years of mathematical preparation. The gap between demonstration-level understanding and production-capable expertise persists, limiting workforce development and constraining participation in AI innovation to a narrow demographic possessing specific educational prerequisites.

Beyond pedagogical challenges, educational deployment of neural network training infrastructure encounters computational constraints absent from production machine learning systems. Cloud training services optimize for heterogeneous workloads where users submit architecturally diverse models with distinct parameter requirements. Educational contexts differ: cohorts of students following standardized curricula construct topologically similar models, particularly in introductory lessons targeting canonical architectures. Traditional infrastructure treats each training session as independent, replicating computation across structurally equivalent networks. Scaling such approaches to institutional deployments supporting hundreds of concurrent users produces unsustainable costs and training latency incompatible with interactive instruction.

The confluence of these challenges - pedagogical inaccessibility of existing tools and computational inefficiency of naive scaling approaches - defines the problem space this research addresses. Effective educational AI platforms require dual innovation: interface designs enabling architectural reasoning without programming prerequisites, and computational strategies exploiting patterns characteristic of educational workloads. Solutions must demonstrate not only pedagogical effectiveness through controlled learning outcome assessment but also technical feasibility through validation of resource efficiency at institutional scale. The absence of platforms satisfying both requirements perpetuates the workforce shortage and limits AI education to populations with substantial prior technical advantages.

Research Objectives

This research pursues three interconnected objectives addressing the identified pedagogical and computational challenges. First, to design and implement an educational platform enabling practical neural network development through visual-interactive methods accessible to secondary school students without programming prerequisites. Second, to develop and validate algorithmic optimizations exploiting architectural convergence in educational workloads, enabling scalable concurrent training without proportional infrastructure costs. Third, to empirically assess learning outcomes comparing visual-interactive instruction against traditional didactic approaches through controlled pedagogical study.

Research Hypotheses

Pedagogical Hypothesis. Visual model construction combined with real-time training observation and AI-guided assistance produces superior conceptual understanding and architectural synthesis capabilities compared to traditional didactic instruction for introductory neural network education. Specifically, students using visual-interactive methods will demonstrate greater gains in post-test performance measuring both terminology comprehension and ability to design architectures

for novel problems, with effects observable across different age groups and baseline preparation levels.

Computational Hypothesis. Detection and exploitation of topological similarity across concurrent training sessions enables sublinear scaling of computational resources relative to user count. Specifically, mechanisms for reusing trained network fragments and consolidating operations across structurally equivalent models will reduce total GPU time and improve hardware utilization compared to naive independent execution, with effectiveness increasing as cohort size grows and architectural convergence intensifies.

Project Type and Research Approach

This work is an **applied research project** in which the central artifact is the implemented software system itself. Neuralese is not a prototype tool built to illustrate a concept - it is a complete, deployable platform whose architectural decisions are the subject of investigation. The pedagogical experiment serves as empirical validation of those decisions, not as the primary research contribution in isolation. The effects measured - comprehension gains, engagement, architectural reasoning ability - depend directly on specific platform properties: the visual graph representation, the granularity of layer-level control, the responsiveness of the training loop, and the behavior of the AI mentor.

Research Contributions

1. **Neuralese:** a complete educational AI engineering environment. A fully implemented client-server platform enabling students to design, train, evaluate, and deploy neural networks through a visual interactive interface without programming prerequisites. Unlike existing tools that either hide model architecture or require professional frameworks, Neuralese exposes the full engineering lifecycle at layer-level abstraction while remaining accessible to secondary school students.
2. **Section Reuse:** weight fragment transfer across topologically similar training sessions. An algorithm that detects architectural equivalence between concurrent training sessions via rough topology hash, partitions trained networks into reusable sections at shape transition boundaries, and initializes subsequent similar sessions from cached weights rather than random initialization. Demonstrated 3.4× reduction in cumulative GPU training time at 16 concurrent users with sublinear scaling behavior.
3. **Fused SuperGraph:** operation-level consolidation of concurrent heterogeneous training sessions. A dynamic graph fusion mechanism that consolidates GPU operations across structurally similar but non-identical models into partially combined models, without requiring identical architectures. Demonstrated 81.6% reduction in CUDA kernel launches and shift of GPU time spent from 18% to 56% in compute-heavy kernels at 16 concurrent users.
4. **Empirical validation of visual-interactive AI instruction.** A longitudinal quasi-experimental study (n = 83, grades 5–7, 21 days, six lessons). Results demonstrate consistent

and substantial learning gains in experimental groups versus controls (Cliff's $\delta \approx 0.56$ for grade 7, $\delta \approx 0.42$ for grade 5), with gains concentrated among initially low-performing students.

Research section

1. Review of Existing Solutions

While universities and research institutions have established rigorous programs in machine learning and deep learning, the accessibility of foundational AI education for younger learners remains constrained by a fundamental gap between pedagogical simplicity and engineering completeness. Existing educational tools fall into distinct categories, each exhibiting characteristic limitations that prevent comprehensive skill acquisition.

- I. Interactive demonstration platforms such as TensorFlow Playground ^[2] provide visualization of neural network behavior through manipulation of hyperparameters and network depth (see Figure 2). These tools successfully illustrate concepts such as activation functions, learning rate dynamics, and decision boundary formation. However, the pedagogical scope remains constrained to synthetic two-dimensional datasets and fixed architectural templates. Users observe pre-configured models rather than construct novel architectures, limiting the transition from conceptual understanding to practical engineering capability.

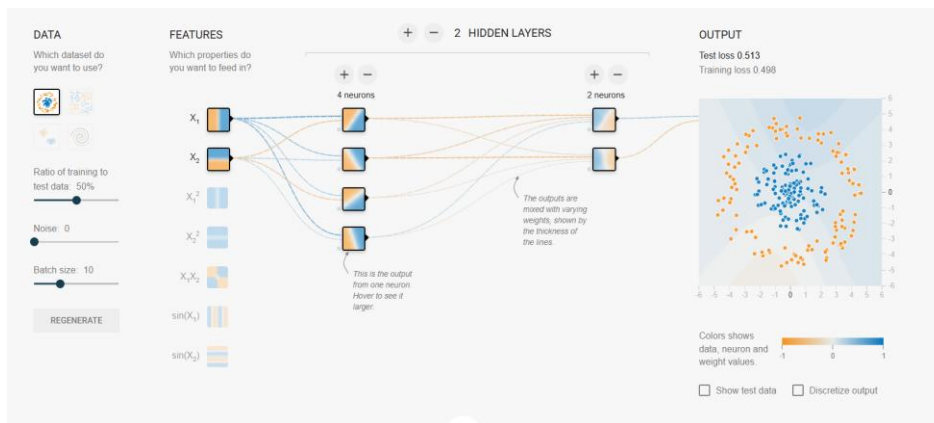


Figure 2. Screenshot from playground.
tensorflow.org

- II. Simplified model-building environments such as Google Teachable Machine ^[3] lower the barrier to entry by abstracting the entire training pipeline into single-click operations (see Figure 3). Users upload image, audio, or text data and receive trained classifiers without exposure to underlying architectural decisions or training dynamics. While this approach demonstrates the practical applicability of machine learning, it operates as a black box that obscures the relationship between model design, data characteristics, and performance

outcomes. The absence of architectural transparency prevents users from developing the diagnostic and optimization skills required for independent model development.

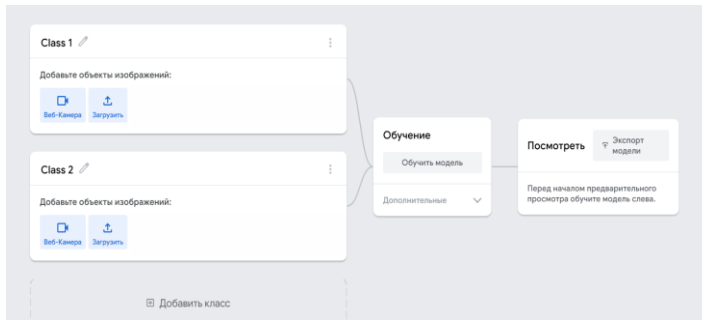


Figure 3. Screenshot from teachablemachine.withgoogle.com

- III. Block-based programming environments adapted for machine learning, including AI Blocks and ML for Kids, apply visual programming paradigms to classification tasks. These platforms leverage the accessibility of drag-and-drop interfaces while introducing basic concepts of supervised learning. However, the predefined template structure restricts architectural flexibility and prevents exploration of layer composition, feature extraction hierarchies, or custom loss functions. The abstraction level remains too high to bridge the gap between block-based interaction and code-level implementation in professional frameworks.
- IV. Professional deep learning frameworks such as PyTorch ^[5] and TensorFlow provide complete control over model architecture, training procedures, and deployment pipelines. These tools represent the industry standard for research and production systems. However, effective utilization requires proficiency in programming languages, linear algebra, calculus, and optimization theory. The cognitive load imposed by simultaneous management of tensor operations, gradient computation, device allocation, and debugging exceeds the capacity of novice learners, particularly those without prior exposure to software engineering or advanced mathematics.

Demonstration tools provide insufficient depth for skill development, while professional frameworks impose prohibitive complexity. The absence of intermediate platforms that preserve engineering fidelity while reducing cognitive overhead creates a barrier to entry that limits participation in AI development to those with substantial technical prerequisites.

A growing body of peer-reviewed literature has examined why this gap persists and why existing tools consistently fail to occupy the middle ground between demonstration and professional practice.

The most precise formulation of the core problem appears in [Tedre et al. \(2021\)](#), who conducted a systematic analysis of machine learning education initiatives across K–12 and identified the inherent design contradiction: **«The more accessible the systems are for novice learners, the more actual ML mechanisms they need to hide from the users - and the less students learn about what goes on under the hood.»** ^[16] This observation directly characterizes the failure mode of platforms such as Teachable Machine and ML for Kids: pedagogical accessibility is achieved at the cost of mechanistic transparency, which is precisely the transparency required for students to transition from

passive observers to active engineers. [Zhou, Van Brummelen, and Lin \(2020\)](#) formalized this as an explicit design challenge, describing it as «**the trade-off between creating transparent systems to provide more comprehensive information and simplifying or 'black-boxing' underlying ML concepts to avoid cognitive overload.**»^[17] These two formulations together define the design space within which all current tools are located - and within which none occupies the intermediate region.

The evidence that existing tools are concentrated at the extremes of this spectrum is well-supported by systematic literature review. [Rizvi, Waite, and Sentance \(2023\)](#) conducted a PRISMA-compliant review of 28 empirical studies of AI teaching and learning in K-12 settings published between 2019 and 2022, and concluded that «mainstream collaborative programming tools used in undergraduate education may be too complex for teaching AI at K-12 level,» with learning outcomes found to be «closely linked with students' prior programming experience.»^[18] This dependency on prior programming background is precisely what prevents professional frameworks such as PyTorch and TensorFlow from serving introductory educational purposes: the cognitive load imposed by simultaneous management of tensor operations, syntax, and mathematical abstractions consistently exceeds novice learners' processing capacity. At the opposite extreme, [Lee and Kwon \(2024\)](#), in a systematic review of AI education in K-12 classrooms from 2018 to 2023, found that «relying solely on AI tools may lead to an oversimplified conceptual understanding of complex AI concepts, potentially overlooking the intricate nuances and ethical considerations that underlie the multifaceted challenges in the field.»^[19] The same review identified a specific instructional failure mode arising from abstracted tools: «If students build AI models and see how they work without understanding the underlying mechanisms (the black box approach), they might struggle to interpret the outcomes effectively and make informed decisions.»^[19]

The structural nature of the black-box problem in ML education is further elaborated by [Tedre et al. \(2021\)](#), who observe that machine learning fundamentally disrupts the computational thinking paradigm that has dominated K–12 computing education for over a decade. In traditional computing education, algorithmic understanding is built on traceable, deterministic step-by-step procedures. ML models violate this pedagogical foundation: «it is practically impossible to trace how a neural network reaches its solution, and the individual 'steps' in the model are not central»^[16] Many educational applications respond to this interpretability problem by hiding the model entirely - a response that eliminates the cognitive challenge but simultaneously eliminates the learning objective. The [Raspberry Pi Foundation's](#) research synthesis of this work notes that «some existing ML tools aim to help learners form mental models of ML, for example through visual representations of how a neural network works. But these explanations are still very complex.»^[20]

The implication is that partial transparency - exposing the graph structure and layer composition without requiring derivation of backpropagation mathematics - is both achievable and educationally necessary.

The academic literature has also identified the specific design principle that resolves this tension: what Goldstein and Papert originally termed the "glass box" approach, enabling learners to look inside the system and ask how it works.^[21] [Broll, Grover, and Babb \(2022\)](#), in research presented at the ACM International Computing Education Research conference, operationalized this principle by developing tools that serve as «a gateway to actually lifting the hood on how such models are

coded,» using layered levels of abstraction to progressively expose implementation detail without overwhelming novice learners. ^[22] The visual graph representation adopted by Neuralese implements this principle directly: the node graph exposes the full computational structure of the model - layer types, connections, data flow, and tensor dimensions - while abstracting away the Python syntax and tensor API that would otherwise dominate learner attention in professional frameworks.

Empirical evidence on the educational consequences of these design choices is accumulating. [Su et al. \(2024\)](#), analyzing 98 AI classroom instruction videos using large language model-assisted content analysis, found that current K-12 AI instruction «insufficiently fosters AI literacy, with only 35.71% addressing higher-level skills such as evaluating and creating AI.» ^[23] AI ethics, a component requiring deep mechanistic understanding to reason about meaningfully, appeared in only 5.1% of instruction analyzed. These figures illustrate the downstream consequence of over-reliance on black-box tools: learners develop surface familiarity with AI applications but lack the architectural understanding necessary for independent system design or critical evaluation. The [Stanford SCALE review of the evidence base on AI in K-12 education \(2026\)](#), synthesizing findings from over 800 academic papers, reached a directly relevant conclusion about tool design: «tools that scaffold reasoning may help support learning, while tools that simply generate answers may reduce the cognitive effort that supports durable skill development.» ^[24]

This distinction between scaffolded reasoning and answer generation precisely characterizes the pedagogical difference between Neuralese's layer-level construction environment and black-box classification tools.

Beyond individual tool design, the broader structural inadequacy of current AI education infrastructure is documented at the system level. [The Stanford HAI AI Index Report \(2026, Chapter 7\)](#) notes that despite widespread exposure to AI tools, formal AI education - defined as instruction in the technical skills required to build AI systems rather than merely use them - «has been slower to take hold» than general computing education, with most countries only beginning to mandate AI-specific curricula as recently as the 2025-26 school year. ^[25] The report's analysis of skill acquisition data, drawn from LinkedIn's [AI Skills Diffusion Index](#), identifies a widening divergence between AI literacy growth, which reflects familiarity with AI-enabled tools, and AI engineering skill growth, which reflects capability to design and build systems. «AI literacy skills show steeper growth, while engineering-oriented skills have increased more modestly.» ^[25] This systemic pattern at the population level mirrors the tool-level pattern observed in the literature: existing educational infrastructure develops consumers of AI technology rather than producers of it.

Taken together, this body of literature establishes three propositions that motivate and validate the design approach implemented in Neuralese. **First**, the accessibility-depth tradeoff is real, structurally embedded in ML's opacity, and not resolvable by incremental improvement of existing tool categories. **Second**, the educational consequences of this tradeoff are measurable and significant - black-box tools produce learners who cannot reason about architectural decisions, while professional frameworks exclude all but those with substantial prior technical preparation. **Third**, the resolution requires a layer-level intermediate environment that exposes mechanistic structure through visual representation rather than programming syntax, implementing the glass-box principle identified in the literature as the theoretically grounded solution to this design problem.

2. Technical Challenge of Scalability in Educational AI Platforms

Educational deployment of neural network training infrastructure encounters computational constraints distinct from conventional cloud machine learning services. Standard distributed training systems optimize for heterogeneous model architectures submitted by independent users, where each training session represents a unique computational graph with distinct parameter requirements. Educational contexts, however, exhibit fundamentally different characteristics that enable alternative optimization strategies.

Client-side execution of neural network training remains infeasible for contemporary architectures. It exceeds the capabilities of integrated graphics processors commonly found in educational computing environments. Browser-based frameworks such as TensorFlow.js provide limited support for small-scale models but suffer from memory constraints, lack of low-precision arithmetic, and inability to utilize dedicated tensor cores. Attempting to execute batch gradient descent on datasets exceeding several thousand examples results in browser memory exhaustion and rendering thread blocking. The absence of asynchronous compilation and kernel fusion further degrades performance relative to native frameworks.

Server-based execution requires careful consideration of resource allocation patterns. A naive implementation that instantiates independent training processes for each user session scales linearly with concurrent users. Profiling of baseline execution with 16 simultaneous training sessions reveals GPU utilization patterns dominated by short-duration CUDA kernel launches with substantial inter-kernel latency. The proliferation of context switches and memory allocation operations reduces effective compute throughput while increasing total training time. For educational deployments projected to support hundreds of concurrent users across multiple institutions, this approach produces unsustainable infrastructure costs and degrades training responsiveness to levels incompatible with interactive pedagogical workflows.

Educational contexts provide a critical optimization opportunity absent from general-purpose training services. Curricula structure learning progressively through standardized exercises targeting specific architectural patterns. Students completing identical lessons construct models with high topological similarity, particularly in introductory sessions focused on canonical architectures such as LeNet-5, ResNet blocks, or simple multi-layer networks. This architectural convergence means that substantial portions of the computational graph remain invariant across users, creating redundant computation when models are trained independently.

The economic implications of this redundancy become apparent when considering institutional deployment scenarios. A cohort of 50 students simultaneously training convolutional classifiers on MNIST represents not 50 unique computational problems but rather *a collection of minor variations* on a shared topology. Traditional approaches execute forward passes, backward passes, and parameter updates independently for each session, despite the fact that identical layer operations with similar dimensions dominate the computational workload. This replication of effort directly translates to

increased GPU-hours, elevated cooling and power costs, and extended training latency that disrupts the pacing of interactive instruction.

Neuralese addresses these constraints through architectural decisions designed specifically for the educational context. The platform implements mechanisms for detecting topological similarity across concurrent training sessions, enabling computational reuse and batch consolidation. Rather than treating each student model as an isolated entity, the system identifies structural equivalence and exploits it to reduce redundant kernel launches and improve GPU occupancy. The technical implementation of these optimizations forms the foundation of Neuralese's ability to scale to institutional deployments while maintaining the responsiveness required for effective pedagogical interaction.

3. Platform Architecture of Neuralese

3.1 System Overview

Neuralese implements a client-server architecture partitioning computational responsibilities according to device capabilities and pedagogical requirements. The client application provides visual model construction, dataset management, and real-time visualization of training dynamics. Computationally intensive operations including gradient computation, parameter updates, and model inference execute on server infrastructure equipped with GPU accelerators. This separation preserves interface responsiveness while enabling access to hardware resources unavailable on typical educational computing devices.

Functionality includes:

- Drag-and-drop, node graph-based visual neural network constructor. Unlike other visual tools, it allows users to edit models at layer-level abstraction.
- Dataset editor for efficient dataset creation and import.
- Scriptable simulation environments for model behavior evaluation. They are scripted in Lua [9] directly within the app.
- Model export to ONNX ^[13], TensorRT ^[14] or standalone executables, allowing usage in other applications such as robotics.
- “Axon” LLM mentor for conscious model development, supporting both dialogue and model construction via MCP-like interface.
- YAML-scriptable interactive tutorials.

3.2 Development process and high-level technologies

The development of Neuralese began in July 2025 as a personal experimental project aimed at creating a visual neural network constructor that would not require programming from the user (see Figure 4).

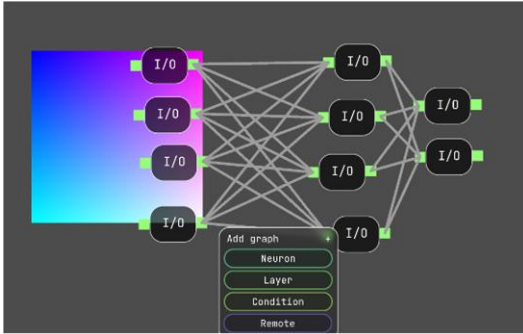


Figure 4. Early prototype of Neuralese. Screenshot by finalists, 2025

Most existing interactive tools of this type are built using JavaScript frameworks based on Chromium/Electron. While convenient for rapid development, such approaches typically result in large application sizes (130–180 MB), high memory consumption (from 200 MB to several gigabytes), and limited rendering performance. For this reason, Godot 4.5^[6] was selected as the core engine for the client application. Godot is a modular and highly optimized engine supporting C++ extensions and multithreading, enabling compact binary size (under 50 MB) and high interface responsiveness.

Initially, Neuralese was conceived purely as a visual AI constructor intended for experimentation with neural network architectures. However, during development it became clear that the visual format significantly increased accessibility. After demonstrating the early prototype to several classmates, we received positive feedback emphasizing that model construction in Neuralese felt interactive rather than observational. Gradually, the idea evolved from a technical experiment into an educational platform.

To determine further development direction, we analyzed existing visual AI learning tools and observed that none fully satisfied educational requirements. Some platforms oversimplify concepts by hiding real model architectures, while others overwhelm learners with excessive mathematical complexity. This analysis became the starting point for designing Neuralese as an environment combining engineering fidelity with pedagogical clarity. Development then focused on constructing its core modules and system architecture.

At an early stage of development, the decision was made to implement the backend using a microservice architecture. This design ensured scalability from the outset: training, inference, project management, and AI mentoring operate as separate services, simplifying updates and future expansion.

The primary technology stack combines Python and Rust, integrated via pyo3. Python (using the asynchronous web framework Sanic^[7]) provides compatibility with modern machine learning

frameworks such as PyTorch and ONNX. Rust modules handle computationally intensive operations including graph serialization and deserialization, dataset processing, model execution, and optimization procedures. This distribution of responsibilities enables high performance without sacrificing development flexibility, and establishes a foundation for future scaling on GPU clusters or distributed server nodes.

3.3 Node graph system

Neuralese’s primary component, the visual neural network constructor (see Figure 5), implements a node-based graph editor enabling drag-and-drop assembly of layer primitives. This design reflects the underlying structure of neural networks: models are fundamentally graphs of operations on input data, making a graph-based editor a more faithful representation than imperative block sequences such as Scratch ^[4].

Users select operations from a categorized palette including dense layers, convolutional layers, pooling operations, activation functions, normalization layers, and tensor manipulation operations, corresponding to real-world layers. Connections between nodes define data flow through the computational graph, with automatic shape inference validating tensor dimension compatibility. Real-time validation provides immediate feedback on architectural errors such as dimension mismatches or cyclic dependencies. The graph representation serializes to JSON format for transmission to the training service and persistent storage.

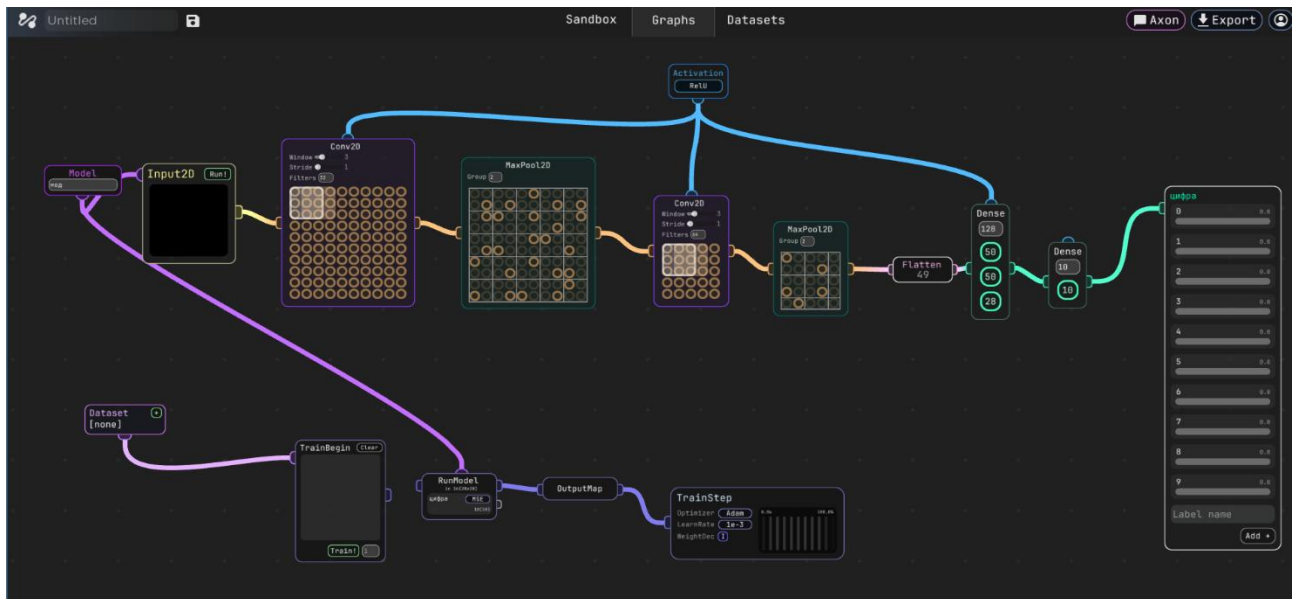


Figure 5. LeNet-like architecture built in Neuralese. Both training and inference graphs are visible. Screenshot by finalists, 2025.

3.4 Dataset editor

The dataset editor was developed as the next major platform component. Dataset management capabilities include a custom spreadsheet-like editor optimized for large-scale tabular data (see Figure

6). The **VirtualTable** component implements view recycling patterns to maintain 120 frames per second rendering on 120Hz displays while handling datasets with tens of thousands of rows. Rather than instantiating visual nodes for all data points, the system maintains a pool of approximately 60 visible row representations and dynamically updates their content as the viewport scrolls. This approach reduces memory consumption by multiple orders of magnitude compared to naive implementations while preserving smooth scrolling interaction.

An integrated command-line interface provides programmatic dataset manipulation for users familiar with data processing operations. The CLI supports filtering by column values, column selection and reordering, data type conversion, row deletion, random shuffling, and statistical summarization. Command syntax follows conventions established by tools such as Pandas and NumPy, facilitating transfer of existing knowledge while avoiding the complexity of full Python environments. All operations execute locally within the client to preserve responsiveness and avoid unnecessary network round-trips.

Dataset management implements block-based storage with compression and incremental synchronization. Large datasets are partitioned into blocks of 512 to 1024 rows, each compressed independently using a two-stage pipeline. Run-length encoding (RLE) exploits repetition in categorical features and sparse numeric data, particularly effective for integer columns with limited cardinality. Output from RLE undergoes ZSTD ^[10] compression providing fast decompression suitable for training data loading. For integer columns with values constrained to the range 0-10, quantization to 4-bit representation precedes RLE, further reducing storage requirements.

Client-side dataset modifications trigger recomputation of SHA-256 ^[11] hashes for affected blocks. Synchronization operations transmit only blocks with hash mismatches to the storage service, minimizing network bandwidth consumption during updates to large datasets. This approach proves particularly effective for datasets undergoing localized edits such as label corrections or feature additions, where the majority of data remains unchanged across versions.

	ID	Имя	Возраст	Город	Дата регистрации	Покупок	Средний
374	100	Ержан	31	Караганда	04.03.2023	13	8338
375	100	Ольга	45	Шымкент	25.09.2023	35	2742
376	100	Айгерин	38	Актау	09.06.2025	17	3165
377	100	Ольга	20	Караганда	29.12.2022	40	4394
378	100	Нурболат	62	Шымкент	17.02.2024	15	1315
379	100	Айсұлу	36	Караганда	11.09.2024	22	1437
380	100	Максим	51	Актау	14.06.2024	15	5257
381	100	Аружан	35	Астана	16.01.2024	44	3761

Figure 6. Sample dataset assembled in Neuralese. Screenshot by finalists, 2025.

3.5 Simulations as evaluation environments

The idea of incorporating simulations emerged at the next stage of platform development as a response to the problem of “abstractness” in machine learning education. Many students perceive neural networks as collections of numbers and graphs, without seeing how model decisions manifest in observable behavior. To make the learning process more meaningful, a visual-physical layer of feedback was introduced - interactive scenes in which a trained model can control an agent, recognize objects, or make decisions in real time.

Neuralese therefore incorporates embedded physics simulations that provide environments where trained models operate autonomously (see Figure 7). By allowing neural networks to drive vehicles along tracks or navigate obstacle courses, the platform establishes a direct connection between architectural decisions and emergent system behavior.

Simulation logic is implemented using **Lua scripts**, with a compiler integrated directly into the client application. This design enables local execution of simulation physics without server communication, preserving responsiveness and reducing infrastructure load. Each simulation defines an observation space representing model inputs and an action space encoding possible outputs. Users connect trained models to these simulation interfaces and observe their behavioral performance at interactive frame rates.

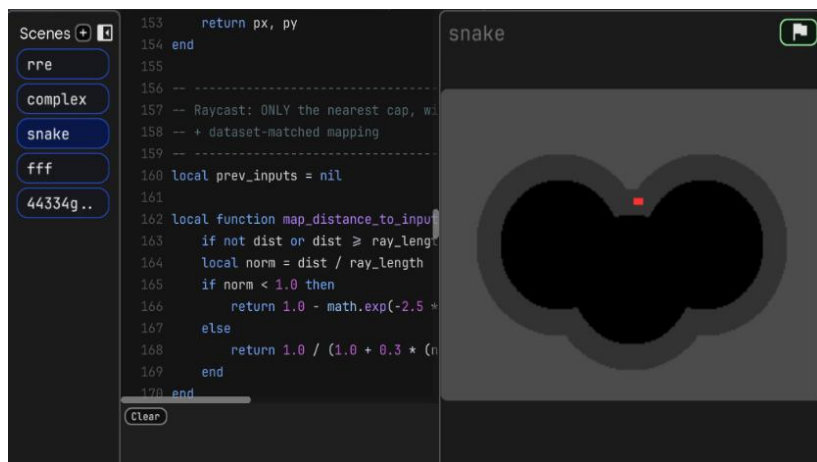


Figure 7. Lua-scripted environment for car driving. Screenshot by finalists, 2025.

3.6 AI Mentor Architecture: Axon

The next stage in the development of Neuralese was the creation of an embedded AI mentor - Axon (see Figure 8).

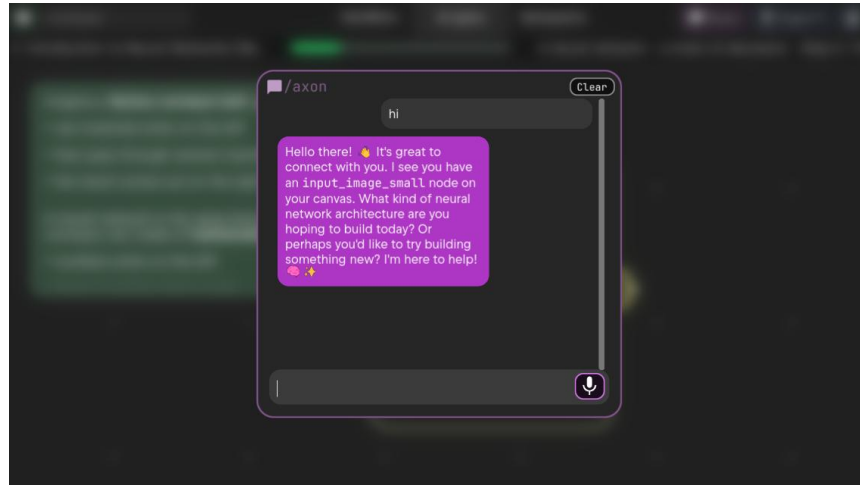


Figure 8. Conversation with Axon. Screenshot by finalists, 2025.

Initially, the idea of integrating an AI mentor emerged as an experiment exploring model interaction protocols. We considered it promising to investigate whether an AI system could assist users in understanding the principles of neural networks directly within the development environment. At this stage, the mentor was conceived primarily as a conversational assistant capable of explaining concepts and responding to user questions.

However, it soon became clear that this functionality carried significantly greater pedagogical importance than initially anticipated. There is currently a noticeable shortage of educators with deep expertise in machine learning and artificial intelligence. Even when institutions are prepared to introduce AI-related coursework, there are often insufficient specialists capable of explaining not only terminology but the underlying logic of model construction. Therefore, we decided to implement Axon as a support layer within the platform - functioning as an individual tutor that assists each student during model construction and training.

For Axon, the Gemini 2.5 Flash API ^[12] was selected due to its balance between generation speed, contextual memory capacity, and reasoning flexibility. Importantly, the architecture itself remains model-agnostic and is not rigidly bound to a specific language model implementation.

After the first integration tests of the AI mentor within Neuralese, it became evident that a simple embedded “chat assistant” was insufficient. In many existing systems, AI serves a purely advisory role - it answers questions but cannot intervene in the user’s workflow.

The design objective therefore expanded: the mentor should not only explain what a layer does, but also be capable of modifying the model itself - adding layers, adjusting parameters, and restructuring connections.

In the initial implementation, all functionality was concentrated within a single agent responsible for both dialogue and architectural modifications. However, this configuration quickly revealed limitations. The model was required to simultaneously maintain conversational tone, reason about architectural structure, and generate precise modification commands. This cognitive overload degraded both the quality of explanations and the reliability of graph transformations.

To resolve this issue, the system was redesigned as a dual-agent architecture (see Figure 9):

- **Narrator** - the communicative and instructional layer. It interacts with the user, provides explanations, maintains conversational context, and formulates strategic suggestions. The Narrator may reason about the model but does not directly modify the graph.
- **Builder** - a service agent operating in a fully stateless and deterministic mode (temperature set to 0.0). It does not generate explanatory text. Instead, it executes structured instructions produced by the Narrator, performing precise graph operations such as adding layers, removing nodes, modifying parameters, or restructuring connections.

This separation ensures architectural stability. The Narrator remains the “teacher,” the Builder acts as the “executor”.

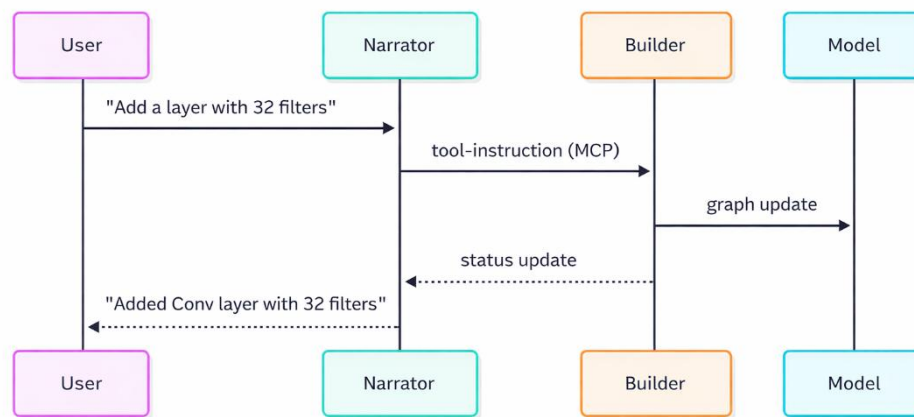


Figure 9. Double-agent architecture diagram. Made by finalists in mermaid.live, 2025.

3.7 Structured Lesson System

Structured lesson system was implemented after the initial pilot study, described in Section 5, to give teachers more control over lesson flow. Formal pedagogical scaffolding is implemented through a scriptable lesson framework enabling instructors to define guided learning sequences. Lessons are authored in **YAML format** describing step-by-step procedures, success criteria, and interface state restrictions. Each step specifies instructional text, visual highlights directing student attention to relevant interface elements, and validation rules checking graph topology or parameter configurations (see Figures 10 and 11). The client application interprets lesson scripts at runtime, constraining available actions to those specified in the current step and providing immediate feedback on task completion.

Validation logic inspects the model graph to verify correct layer types, connection patterns, and parameter values. For example, a lesson on convolutional networks may require students to construct a graph containing Conv2D layers with specific kernel sizes followed by MaxPooling operations. The system checks graph topology by traversing node connections and validating layer configurations against lesson requirements. Upon successful completion, visual feedback including animations and interface unlocks signals progression to the next step.

```

lesson_title: "Neuralese Basics"

flow:
  - step welcome:
    - require:
      teacher_lock: {}
    - explain:
      text:
        - "👋 Today we'll learn:"
        - "1. How to create [b]blocks[/b]"
        - "2. How to connect them with [b>wires[/b>"
        - "3. And why do they sometimes [b]not connect[/b>"
        - ""
        - "Good luck! 🍀"
      wait: next

```

Figure 10. Sample lesson step. Made by finalists in carbon.now.sh, 2025.

The **teacher_lock** mechanism enables classroom synchronization by preventing students from advancing beyond designated checkpoints until the instructor authorizes progression. This feature supports paced instruction where the teacher provides explanations between lesson segments while ensuring all students complete critical steps before proceeding. The combination of automated validation and manual pacing control balances autonomous learning with structured pedagogical oversight.

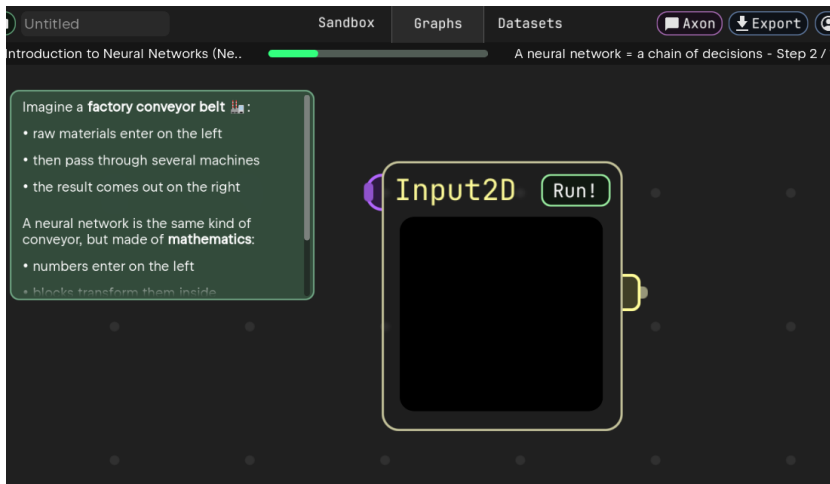


Figure 11. Step in introductory Neuralese lesson. Screenshot by finalists, 2025.

3.8 Model Export: Yolk-Protein

Neuralese provides model export capabilities producing deployment-ready artifacts from trained models. The export system, designated Yolk-Protein, generates standalone executables or standardized model formats suitable for integration into external applications. **Yolk** refers to the precompiled runtime engine implemented in Rust, while **Protein** denotes the serialized model weights and metadata package.

Export to executable format produces self-contained binaries bundling the Yolk inference engine with model weights. These executables load input data, perform forward passes, and return predictions without requiring external dependencies beyond system libraries. The Rust

implementation provides fast compilation times, with typical 100-megabyte models assembling into executables in approximately 0.04 seconds. Exported models support ONNX and TensorRT target formats, enabling deployment to edge devices, mobile platforms, and production inference services.

This export capability completes the development cycle by enabling students to produce deployable artifacts from educational projects. Models trained in Neuralese become components of larger systems rather than remaining confined to the learning environment, demonstrating the practical applicability of acquired skills.

4. Initial pilot study

4.1 Pilot Lesson Design and Results

Prior to the longitudinal study, a preliminary pilot lesson was conducted in December 2025 at the authors' school during grade 8 computer science classes. This exploratory session served two purposes: to test the viability of the visual-interactive approach and to identify platform limitations requiring resolution before sustained classroom integration.

Twenty students were randomly assigned to an experimental group ($n = 10$) using Neuralese Beta 0.9 and a control group ($n = 10$) receiving traditional instruction via presentations. Pre- and post-tests measuring understanding of fundamental AI concepts were administered to both groups, along with a Likert-scale interest survey. Effect sizes were estimated using Cohen's d , appropriate for small samples.

The experimental group demonstrated a 27 percentage point improvement in conceptual understanding versus 14 percentage points in the control group (Cohen's $d \approx 0.5$, medium effect). Interest scores showed a comparable differential: +32% in the experimental group versus +12% in the control group ($d \approx 0.61$). These preliminary results confirmed that the visual-interactive approach yielded measurable educational benefits, supporting the decision to proceed with longitudinal implementation.

4.2 Limitations Identified and Engineering Consequences

The pilot also exposed two critical limitations. First, without a structured lesson system, students diverged rapidly in pace and graph state, making classroom-wide instruction difficult to manage. This motivated the YAML-scriptable lesson framework and synchronization mechanism described in Section 3.7. Second, projecting the platform to institutional scale - estimated at 50 schools with peak loads of approximately 1,000 concurrent training sessions - revealed that the existing execution model would scale linearly with user count, producing unsustainable GPU costs and training latency incompatible with interactive instruction.

The key insight enabling both optimizations was that educational workloads are structurally unlike production machine learning: cohorts of students following the same curriculum inevitably construct topologically similar models, creating systematic redundancy in naive independent

execution. This observation - that thousands of students in identical lessons represent not thousands of unique computational problems but variations on shared topologies - became the theoretical foundation for Section Reuse and Fused SuperGraph, developed and validated in the following section.

5. Algorithmic Optimizations for Concurrent Training

5.1 Section Reuse: Exploiting Architectural Similarity

Educational deployment scenarios exhibit substantial architectural convergence across concurrent training sessions. Students following structured curricula construct models with high topological similarity, differing primarily in hyperparameter choices rather than fundamental layer arrangements. This pattern enables optimization through selective reuse of previously trained network fragments, reducing redundant computation without compromising model independence.

The **Section Reuse** mechanism (see Figures 12 and 13) operates by first computing a rough topology hash (RTH) for each submitted model. The RTH derives from layer types and ordering but deliberately excludes precise dimensional parameters, enabling matching of structurally equivalent networks despite minor variations in feature counts or kernel sizes. Models with identical RTH values share the same high-level architecture, making them candidates for computational reuse.

Upon completion of training for a model, the system performs automatic graph partitioning, decomposing the network into sections bounded by shape transitions. A section represents a contiguous sequence of operations whose output tensor dimensions differ from the input dimensions, typically encompassing a feature extraction stage or classification head. For LeNet-5, partitioning produces four sections: Conv2D(16) with ReLU activation, Conv2D(32) with ReLU, Flatten followed by Dense(128) with ReLU, and a final Dense(10) output layer.

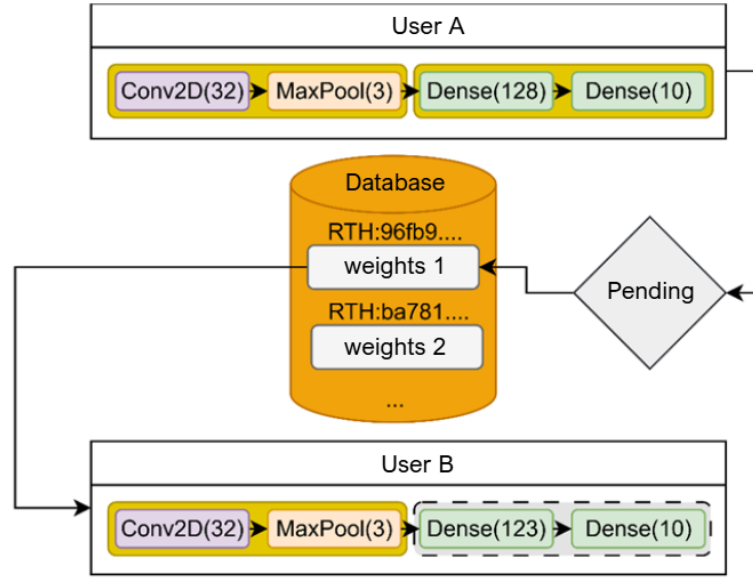


Figure 12. Section Reuse architecture, given user A and user B. Made by finalists in draw.io, 2026.

Each section is serialized with trained weights, input and output tensor signatures, and layer configuration metadata. These serialized sections initially populate a Pending Cache maintained in server memory and indexed by RTH. When a subsequent user initiates training with a matching RTH, the system attempts to locate compatible sections from the Pending Cache. Compatibility requires exact type matching and shape compatibility, with partial matches handled through mathematical approximation.

For dense layers, dimension expansion employs weight matrix extension. If a cached section contains Dense(128) and the new model requires Dense(256), the system constructs an expanded weight matrix by retaining the first 128 columns from the cached weights and initializing the remaining 128 columns with scaled Gaussian noise. The scaling factor is chosen to preserve activation variance, maintaining gradient flow characteristics during subsequent training. Formally, the extended weight matrix W' combines cached weights W with noise N :

$$W' = [W \mid \alpha \cdot N]$$

where α controls variance preservation. Convolutional layers undergo similar expansion, with additional filters generated as scaled combinations of existing filters. If a Conv2D layer expands from 32 to 64 output channels, the 32 additional filters are initialized as weighted sums of the original 32 filters plus controlled noise, maintaining statistical properties of feature map distributions.

Upon successful section matching and initialization, the new model begins training with parameters partially inherited from prior sessions rather than random initialization. This substantially accelerates convergence, as the network enters optimization from a state already capturing relevant feature hierarchies. After training completion, the refined sections are persisted to MinIO object storage, becoming available for future users with matching RTH values. The Pending Cache thereby

transitions from short-term reuse within a class session to long-term knowledge sharing across cohorts.

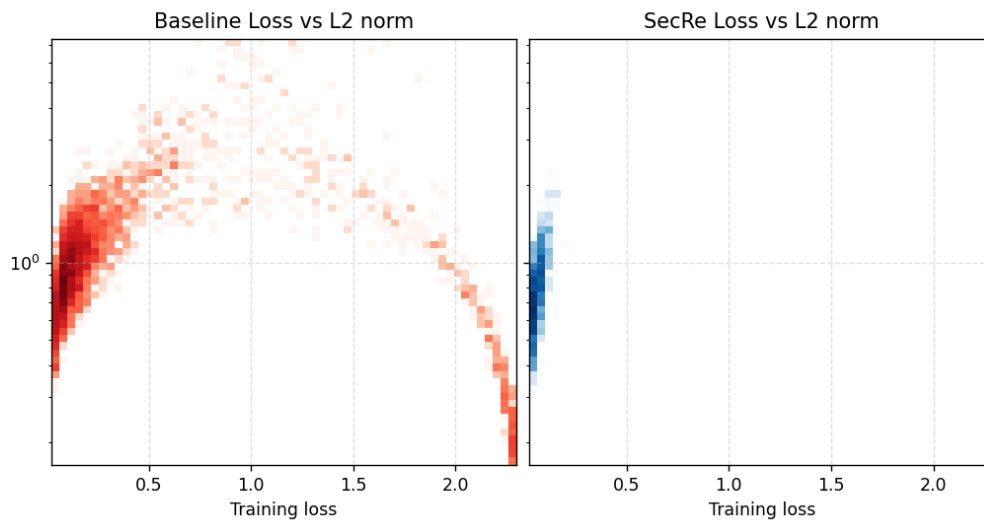


Figure 13. Faster convergence with Section Reuse. Made by finalists in matplotlib, 2026.

Profiling with 1 to 16 concurrent users demonstrates sublinear scaling of total GPU time (Figure 14). In baseline execution without Section Reuse, training 16 models with batch size 64 requires approximately 43.87 seconds of cumulative GPU time, growing nearly linearly with user count. With Section Reuse enabled, the same workload completes in 13.02 seconds, corresponding to a 3.4-fold acceleration. The mechanism proves most effective when multiple students train similar architectures within short time windows, a common pattern in classroom-based learning.

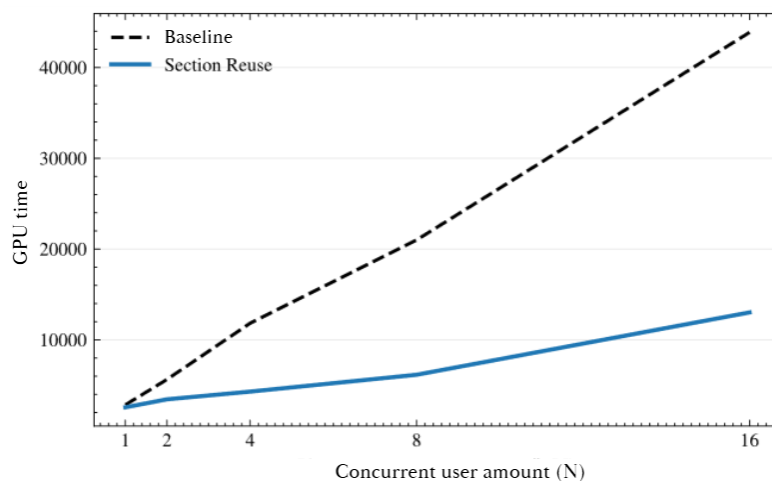


Figure 14. T scaling comparison with growing number of users. Made by finalists in matplotlib, 2026.

5.2 Fused SuperGraph: Consolidated Execution

While Section Reuse reduces training time for individual models, concurrent execution of multiple sessions still suffers from GPU underutilization due to excessive kernel launch overhead and poor kernel granularity. Profiling of 16 parallel training sessions reveals that GPU occupancy remains low despite abundant computational work, as the system spends significant time transitioning between independent training contexts, as shown in Figure 15. Each model executes forward passes, backward passes, and optimizer steps as separate kernel sequences, producing a fragmented execution timeline with frequent idle periods.

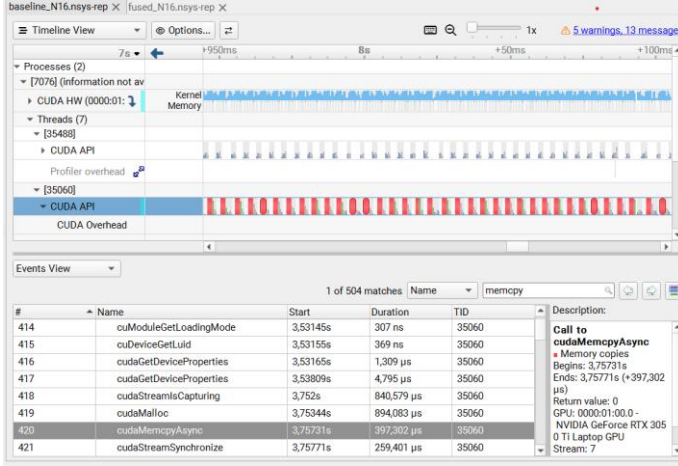


Figure 15. NSIGHT ^[15] profiling results. Screenshot by finalists, 2026.

Our initial approach applied standard super-batching, combining input batches across concurrent sessions to increase effective batch size. This required identical model architectures - a condition that production ML deployments can enforce but educational contexts cannot. Students following the same lesson nonetheless diverge in layer dimensions, activation choices, and architectural mistakes, making strict topological identity rare in practice. This heterogeneity ruled out naive super-batching and motivated an alternative that operates at the granularity of individual operations rather than entire models, preserving logical independence between student sessions while enabling physical consolidation at the kernel level.

The **Fused SuperGraph (FSG)** mechanism addresses this by consolidating operations rather than entire models. Models sharing the same rough topology hash are identified as candidates for fusion. Rather than executing each model's operations independently, the system constructs a unified computational graph incorporating all models within an RTH bucket. Operations are merged at the granularity of individual layers, enabling partial fusion even when models differ in some aspects of their architecture (see Figure 16).

When a new training session joins an existing RTH bucket, the system performs ahead-of-time compilation to regenerate the fused graph. Each layer type in the computational graph is analyzed for fusibility. For example, convolutional operations with identical kernel sizes but different channel counts are transformed into grouped convolutions, where the total output channel count equals the sum of individual model requirements; dense layers are converted to block-diagonal matrix

multiplications, where each model's fully connected operation becomes a submatrix within a larger combined operation. This fusion is applied uniformly across all fusible layer types. The resulting TorchScript module encapsulates the fused computation, enabling optimized execution through PyTorch's JIT compiler.

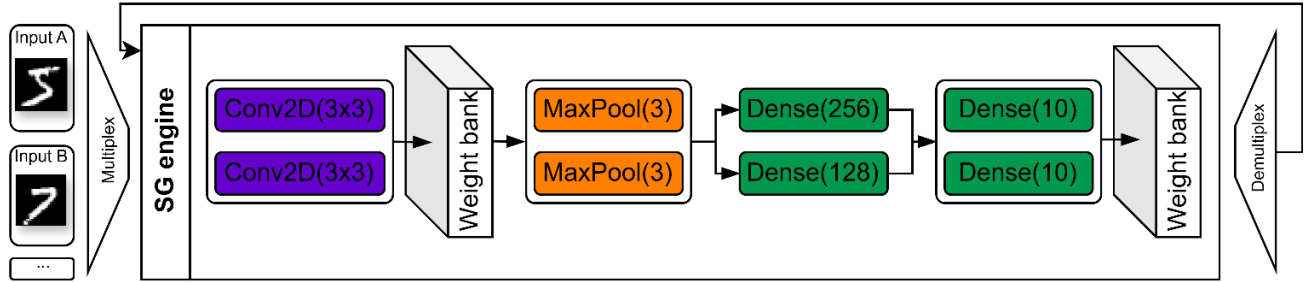


Figure 16. LeNet architectures of users A and B when fused in FSG. Made by finalists in draw.io, 2026.

During training execution, input batches from all models are concatenated along the batch dimension. If each model processes batch size 32, a fused graph serving two models operates on effective batch size 64. This concatenated input propagates through the fused operations, with grouped convolutions and block-matrix multiplications producing outputs that are subsequently partitioned back to individual model contexts for loss computation and gradient calculation.

A critical component enabling efficient parameter updates is the **weight bank** mechanism. Rather than maintaining separate parameter tensors for each model, all weights for a given layer type and training step are stored in a single contiguous tensor. Each model is assigned a parameter range within this tensor, for example indices 0-99 for the first model and 100-199 for the second. Forward and backward passes operate on the unified tensor, with gradient routing ensuring that parameter updates affect only the appropriate ranges. This eliminates the need for repeated tensor concatenation during training, substantially reducing memory allocation overhead.

Profiling results with 16 concurrent users demonstrate the effectiveness of FSG optimization. Baseline execution launches over 160,000 CUDA kernels with average duration of approximately 0.4 milliseconds, indicative of poor GPU utilization. FSG reduces kernel count to approximately 30,000, an 81.6% reduction, while increasing average kernel duration to 6.5 milliseconds. This shift reflects consolidation of many small operations into fewer compute-intensive kernels. Total GPU time decreases from 14.2 seconds to 11.6 seconds, an 18% reduction, while the proportion of time spent in computationally heavy kernels increases from 18% to 56%, which is demonstrated in Figure 17.

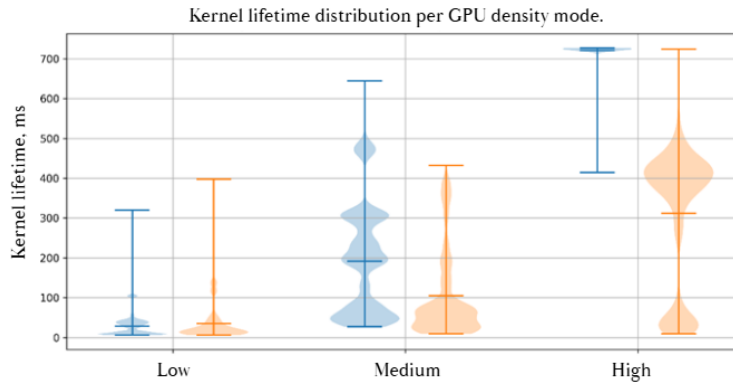


Figure 17. Kernel lifetime distributions. Made by finalists in matplotlib, 2026.

The performance improvement derives from better hardware utilization rather than fundamental algorithmic changes. By launching fewer, larger kernels, FSG reduces the fraction of execution time consumed by driver overhead, memory allocation, and kernel launches. The GPU spends more time performing actual computation and less time managing kernel launches. This optimization proves particularly effective for educational deployments where dozens of structurally similar models train concurrently, a scenario rarely encountered in production machine learning systems but commonplace in classroom environments.

6. Pedagogical validation

6.1 Study Design and Implementation

Following technical validation of platform stability, Neuralese underwent longitudinal pedagogical evaluation embedded within regular curriculum at a secondary school (Figures 18-21). The study employed quasi-experimental design comparing learning outcomes between students using Neuralese (experimental groups) and those receiving traditional instruction (control groups) over 21 days encompassing six lessons. The sample comprised 83 students distributed across five groups: grade 7 experimental (n=20) and control (n=24), grade 5 experimental groups 1 and 2 (n=11, n=13) and control (n=15).

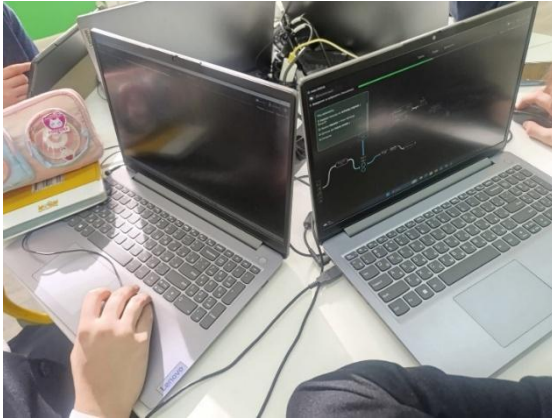


Figure 18. 7th grade students completing introductory tutorial. Photo taken by finalists, 2026.

Control groups received teacher-led instruction using presentations and conceptual diagrams covering neurons and weights, forward and backward propagation, activation functions, layer types, architectural principles, and convolutional networks. Experimental groups learned identical content through Neuralese platform interaction, constructing architectures with the visual graph editor, managing datasets, configuring training parameters, and observing real-time learning dynamics. The AI mentor Axon provided on-demand explanations and architectural guidance while instructors managed pacing through **teacher_lock** controls. Assessment employed pre-test (18 points) and post-test (20 points with synthesis component) instruments measuring terminology retention, conceptual understanding, and ability to use architectures for problems. Both instruments were reviewed for content validity prior to administration by a senior informatics educator independent of the study authors, who also proctored all post-test sessions across conditions.

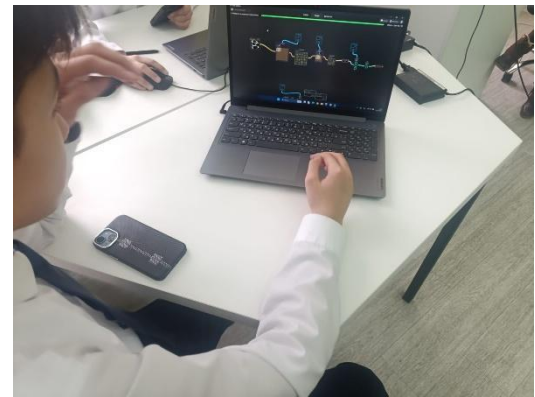
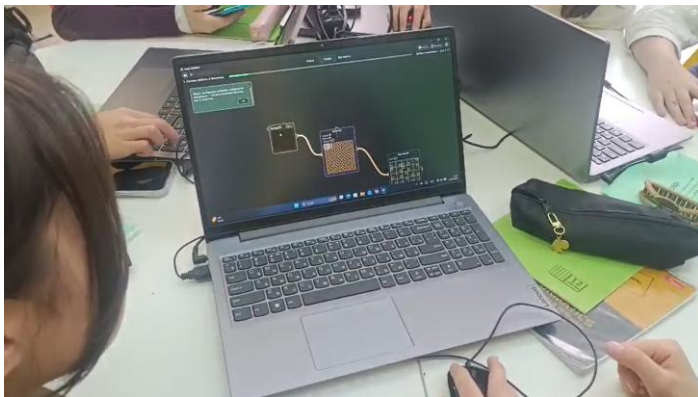


Figure 19-20. Users working on LeNet-5 architectures. Photos taken by finalists, 2026.

6.2 Results and Effect Sizes

Group	Pre-test	Post-test	Growth (points)	Growth (%)
5-EG-1	3.45	9.3	+5.85	+27
5-EG-2	3.38	8.31	+4.93	+22
5-CG	3.67	6.64	+2.97	+12
7-EG	5.05	11.2	+6.15	+29
7-CG	5.71	8.6	+2.89	+11

Because pre-test and post-test used different maximum scores (18 and 20), learning gains are reported as percentage-point changes relative to scale maxima.

Pre-test results established comparable baseline knowledge between conditions within grade levels. Grade 5 groups exhibited low initial scores (experimental: 3.45, 3.38; control: 3.67) reflecting minimal prior ML exposure. Grade 7 experimental group (5.05) started slightly below control (5.71), ensuring no pre-existing advantage. Post-test results demonstrated substantial differential gains. Grade 7 experimental groups achieved a mean score of 11.2/20, corresponding to an improvement of 29 percentage points, compared to 11 percentage points in control groups. Grade 5 experimental groups reached 8.3–9.3/20, corresponding to improvements of 22–27 percentage points.

Distributional analysis revealed that experimental gains derived primarily from improvement among initially low-performing students, indicating barrier reduction rather than selective acceleration of advanced learners. Effect size estimation via Cliff's delta yielded $\delta \approx 0.56$ (grade 7, medium-large effect) and $\delta \approx 0.42$ (grade 5, large effect), confirming robust educationally meaningful differences.

5. Ниже напиши, или нарисуй, схему того как нейросеть распознает рукописные цифры с картинок.

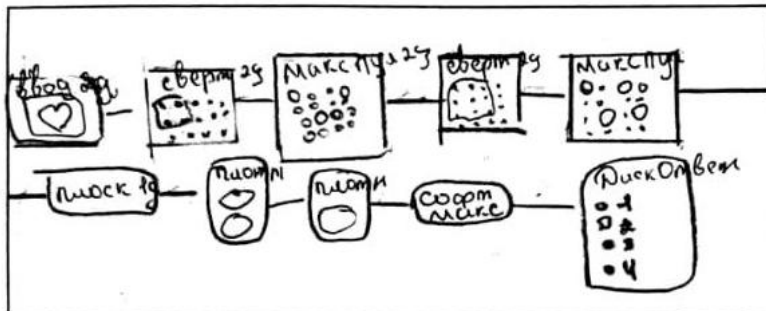


Figure 21. “Draw or describe an architecture of neural network that recognizes handwritten digits from pictures”. Student’s answer on architecture synthesis question. Scanned by finalists, 2026.

6.3 Interpretation

Results provide evidence that visual-interactive instruction with AI mentoring produces superior outcomes compared to traditional didactic approaches for introductory neural network education. Consistency across grade levels despite substantial developmental differences suggests benefits arise from fundamental pedagogical mechanisms rather than age-specific factors. The concentration of gains among initially struggling students indicates effective barrier reduction, with interactive construction and individualized AI assistance enabling self-paced progression. The success of grade 5 students in achieving meaningful understanding despite minimal mathematical background demonstrates viability of introducing AI literacy earlier than currently standard in most curricula. While quasi-experimental design and absence of long-term follow-up limit causal inference strength, the magnitude and consistency of observed effects across multiple cohorts support core hypothesis validity regarding visual-interactive instruction accessibility and comprehension benefits.

Conclusion

This research addressed the dual challenge of pedagogical inaccessibility and computational inefficiency in introductory artificial intelligence education. The objective was not merely to create another educational tool, but to design, implement, and empirically validate an integrated platform that preserves the full engineering lifecycle of neural network development while remaining accessible to secondary school students without programming prerequisites. The results obtained confirm the validity of the proposed approach on both educational and technical dimensions.

From a pedagogical perspective, the study demonstrates that visual-interactive model construction combined with real-time observation and AI-guided assistance leads to significantly superior learning outcomes compared to traditional didactic instruction. Longitudinal educational evaluation across multiple cohorts revealed consistent and substantial gains in conceptual understanding, architectural reasoning, and synthesis ability among students using Neuralese. The observed effect sizes, particularly among learners with initially low baseline performance, indicate that the platform effectively reduces entry barriers rather than selectively benefiting already advanced students. These findings support the pedagogical hypothesis that architectural reasoning can be meaningfully developed prior to formal mastery of programming syntax and advanced mathematics, enabling earlier and more inclusive AI literacy.

From a systems and infrastructure perspective, the research validates the computational hypothesis that educational workloads exhibit exploitable structural regularities distinct from production machine learning environments. The Section Reuse mechanism demonstrated a 3.4-fold reduction in cumulative training time by leveraging architectural similarity across concurrent sessions, while the Fused SuperGraph optimization reduced CUDA kernel launches by over 80% and improved effective GPU utilization. These results confirm that sublinear scaling with respect to user count is achievable in classroom scenarios, enabling institutional deployment without proportional growth in infrastructure costs. Importantly, these optimizations preserve model independence and pedagogical flexibility, allowing students to experiment with architectural variations while benefiting from shared computational execution.

The integration of these pedagogical and technical contributions within a unified platform constitutes the primary novelty of Neuralese. Unlike existing tools that either oversimplify model construction or impose prohibitive technical complexity, Neuralese exposes authentic neural network engineering processes through interface design rather than conceptual abstraction. The inclusion of simulation-based evaluation environments, structured lesson scripting, AI-assisted co-modeling via a dual-agent mentor architecture, and deployment-ready model export completes the learning cycle from conceptual design to functional application. This positions Neuralese not as a demonstrative learning aid, but as an intermediate bridge between introductory education and professional machine learning frameworks.

The practical implications of this work extend beyond the immediate experimental setting. Neuralese provides a viable foundation for integrating AI system development into secondary education curricula, supporting early formation of engineering intuition and problem-solving skills.

The demonstrated scalability enables deployment at the level of schools, universities, and extracurricular centers without excessive hardware investment. Furthermore, the platform's export capabilities allow student projects to transition into real-world applications, reinforcing motivation and demonstrating the societal relevance of acquired competencies.

Several limitations warrant consideration. The quasi-experimental design and limited duration of the longitudinal study constrain claims regarding long-term knowledge retention and transfer to code-based frameworks. Future work should include extended follow-up assessments, comparative studies against hybrid visual-code environments, and evaluation of progression from Neuralese to professional toolchains. On the technical side, further optimization of dynamic fusion strategies and expansion to reinforcement learning workloads present promising directions.

In summary, this research demonstrates that meaningful AI system development education can be made accessible, scalable, and pedagogically effective through deliberate alignment of interface design, instructional methodology, and computational architecture. Neuralese represents a step toward democratizing participation in AI engineering by lowering entry barriers without diluting technical authenticity, addressing a critical need in contemporary education systems facing rapid technological transformation.

References

- [1] Maslej N., et al.; Artificial Intelligence Index Report 2025: Policy Highlights; Stanford, CA; Stanford HAI; 2025; 12 p.
- [2] TensorFlow Team; TensorFlow Playground; Online; Google; 2026; n.p.
- [3] Carney M., Webster B., Alvarado I., Phillips K., Howell N., Griffith J., Jongejan J., Pitaru A., Chen A.; Teachable Machine: Approachable Web-Based Tool for Exploring Machine Learning Classification; New York; ACM; 2020; pp. 1–8.
- [4] Resnick M., Maloney J., Monroy-Hernández A., Rusk N., Eastmond E., Brennan K., Millner A., Rosenbaum E., Silver J., Silverman B.; Scratch: Programming for All; New York; ACM; 2009; Vol. 52, No. 11; pp. 60–67.
- [5] Paszke A., Gross S., Massa F., Lerer A., Bradbury J., Chanan G., Killeen T., Lin Z., Gimelshein N., Antiga L., Desmaison A., Kopf A., Yang E., DeVito Z., Raison M., Tejani A., Chilamkurthy S., Steiner B., Fang L., Bai J., Chintala S.; PyTorch: An Imperative Style, High-Performance Deep Learning Library; Red Hook, NY; Curran Associates, Inc.; 2019; NeurIPS 2019; pp. 8024–8035.
- [6] Godot Engine Community; Godot Engine Documentation (Version 4.5); Online; Godot Engine Project; 2025; n.p.
- [7] Sanic Community; Sanic User Guide; Online; Sanic Framework; 2026; n.p.
- [8] PyTorch Contributors; TorchScript (PyTorch JIT) Documentation; Online; PyTorch.org; 2025; n.p.
- [9] Ierusalimsky R., de Figueiredo L.H., Celes W.; Lua 5.4 Reference Manual; Online; Lua.org; 2025; n.p.
- [10] Collet Y., Kucherawy M.; Zstandard Compression and the 'application/zstd' Media Type; Fremont, CA; RFC Editor (IETF); 2021; RFC 8878; pp. 1–11.
- [11] National Institute of Standards and Technology; Secure Hash Standard (SHS); Gaithersburg, MD; NIST; 2015; FIPS PUB 180-4; pp. 1–36.
- [12] Google; Gemini API Documentation; Online; Google AI for Developers; 2026; n.p.
- [13] ONNX Community; Open Neural Network Exchange (ONNX) Specification; Online; ONNX Project; 2026; n.p.
- [14] NVIDIA Corporation; NVIDIA TensorRT Developer Guide; Santa Clara, CA; NVIDIA; 2024; v10.x; n.p.
- [15] NVIDIA Corporation; Nsight Systems User Guide; Santa Clara, CA; NVIDIA; 2026; n.p.
- [16] Tedre M., Toivonen T., Kahila J., Vartiainen H., Valtonen T., Jormanainen I., Pears A.; Teaching Machine Learning in K–12 Classroom: Pedagogical and Technological Trajectories for Artificial Intelligence Education; IEEE Access; 2021; Vol. 9; pp. 110558–110572. doi:10.1109/ACCESS.2021.3097962
- [17] Zhou X., Van Brummelen J., Lin P.; Designing AI Learning Experiences for K-12: Emerging Works, Future Opportunities and a Design Framework; arXiv preprint arXiv:2009.10228; 2020.
- [18] Rizvi S., Waite J., Sentance S.; Artificial Intelligence Teaching and Learning in K-12 from 2019 to 2022: A Systematic Literature Review; Computers and Education: Artificial Intelligence; 2023; Vol. 4; 100145. doi:10.1016/j.caeai.2023.100145

- [19] Lee S.J., Kwon K.; A Systematic Review of AI Education in K-12 Classrooms from 2018 to 2023: Topics, Strategies, and Learning Outcomes; *Computers and Education: Artificial Intelligence*; 2024; Vol. 6; 100211. doi:10.1016/j.caeai.2024.100211
- [20] Raspberry Pi Foundation; *The Machine Learning Effect: Magic Boxes and Computational Thinking 2.0*; Cambridge; Raspberry Pi Foundation Research Seminars; 2021.
- [21] Goldstein I., Papert S.; *Artificial Intelligence, Language, and the Study of Knowledge*; *Cognitive Science*; 1977; Vol. 1, No. 1; pp. 84–123.
- [22] Broll B., Grover S., Babb D.; *Beyond Black-Boxing: Building Intuitions of Complex Machine Learning Ideas Through Interactives and Levels of Abstraction*; *Proceedings of the 2022 ACM Conference on International Computing Education Research*; 2022; pp. 21–23. doi:10.1145/3501709.3544273
- [23] Su J., et al.; *Analyzing K-12 AI Education: A Large Language Model Study of Classroom Instruction on Learning Theories, Pedagogy, Tools, and AI Literacy*; *Computers and Education: Artificial Intelligence*; 2024. doi:10.1016/j.caeai.2024.100322
- [24] Stanford SCALE Initiative; *The Evidence Base on AI in K-12: A 2026 Review*; Stanford, CA; Stanford University; 2026.
- [25] Mahnken K., et al.; *AI Index Report 2026, Chapter 7: Education*; Stanford, CA; Stanford HAI; 2026. Available at: <https://hai.stanford.edu/ai-index/2026-ai-index-report/education>